# شرح سطر بسطر ١٢ — حالة الرئيسية المشتقة من البيانات (HomeState)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الرئيسية لا تحتوي أي رقم أو اسم ثابت. كل ما تعرضه يُشتقّ من قاعدة البيانات والكتالوج للملف النشط: طالب جديد ← حالة «أول يوم» (شاشة 28)، وبعد أول نشاط ← الحالة العادية بنجومه وسلسلته وتقدّمه (شاشة 03).

`HomeState` كائن غير قابل للتغيير يصف «ما يجب أن تعرضه الشاشة». يُبنى بدالة صرفة (`HomeState.derive`) فتُختبر بلا واجهة (16 اختبارًا في `home_state_test.dart`).

`homeStateProvider` يجمع المصادر الخمسة (الملف والكتالوج والتقدّم والمحاولات والإتقان) في `AsyncValue`؛ وأي تغيير في القاعدة يعيد حساب الحالة فتتحدّث الشاشة وحدها.

## lib/features/home/home_state.dart — اشتقاق حالة الرئيسية

`lib/features/home/home_state.dart`

**الأسطر 1–3**

```
1  import '../../core/db/app_database.dart';
2  import '../../core/utils/streak.dart';
3  import '../catalog/catalog_models.dart';
```

**ماذا يفعل؟** استيراد القاعدة (لأنواع الصفوف والتعدادات) وأداة السلسلة ونماذج الكتالوج.

**لماذا بهذه الطريقة؟** الدالة صرفة لكنها تستعمل هذه الأنواع.

**الأسطر 5–6**

```
5  /// عتبة الإتقان تحت الحدّ يُعدّ المفهوم «يحتاج إلى مراجعة».
6  const double kReviewMasteryThreshold = 0.6;
```

**ماذا يفعل؟** ثابت `kReviewMasteryThreshold = 0.6`.

**لماذا بهذه الطريقة؟** المفهوم الذي إتقانه أقل من 0.6 «يحتاج إلى مراجعة». ثابت واحد يغيّره المصمّم بسهولة.

**الأسطر 8–9**

```
8  /// أقصى عدد بطاقات مراجعة تُعرض في الرئيسية.
9  const int kMaxReviewItems = 3;
```

**ماذا يفعل؟** ثابت `kMaxReviewItems = 3`.

**لماذا بهذه الطريقة؟** لا نغرق الرئيسية ببطاقات مراجعة كثيرة.

الأسطر 11–18

```
/// درس مع سياقه (مادته ووحدته) ونسبة تقدّم الطالب فيه.
class LessonRef {
  const LessonRef({
    required this.lesson,
    required this.subject,
    required this.unit,
    required this.progress,
  });
```

**ماذا يفعل؟** صنف `LessonRef`: درس مع مادته ووحدته ونسبة التقدّم.

**لماذا بهذه الطريقة؟** البطاقة الخضراء تعرض «المادة · الوحدة» و«الدرس» ونسبة التقدّم، فجمعناها في كائن واحد بدل البحث في الكتالوج داخل الواجهة.

الأسطر 20–26

```
  final CatalogLesson lesson;
  final CatalogSubject subject;
  final CatalogUnit unit;

  /// 0..1
  final double progress;
}
```

**ماذا يفعل؟** الحقول: الدرس والمادة والوحدة والنسبة (0..1).

**لماذا بهذه الطريقة؟** غير قابلة للتغيير.

الأسطر 28–34

```
class SubjectProgress {
  const SubjectProgress({
    required this.subject,
    required this.completed,
    required this.total,
    required this.started,
  });
```

**ماذا يفعل؟** صنف `SubjectProgress` لتقدّم مادة.

**لماذا بهذه الطريقة؟** لبطاقات «موادي».

الأسطر 36–41

```
  final CatalogSubject subject;
  final int completed;
  final int total;

  /// هل بدأ الطالب أي درس في هذه المادة؟
  final bool started;
```

**ماذا يفعل؟** الحقول: المادة وعدد المكتمل والإجمالي وهل بدأت.

**لماذا بهذه الطريقة؟** `started` يفرّق بين «لم تبدأ» و«0%».

الأسطر 43–45

```
  /// نسبة تقدّم المادة = الدروس المكتملة ÷ دروس المادة.
  double get fraction => total == 0 ? 0 : completed / total;
}
```

**ماذا يفعل؟** `fraction`: المكتمل ÷ الإجمالي (0 إن لم توجد دروس).

**لماذا بهذه الطريقة؟** نسبة التقدّم تُحسب ولا تُخزَّن؛ وحماية القسمة على صفر.

الأسطر 47–52

```
class ReviewItem {
  const ReviewItem({
    required this.concept,
    required this.lesson,
    required this.mastery,
  });
```

**ماذا يفعل؟** صنف `ReviewItem` لعنصر مراجعة.

**لماذا بهذه الطريقة؟** مفهوم ضعيف مع درسه ونسبة إتقانه.

الأسطر 54–59

```
  final CatalogConcept concept;
  final CatalogLesson lesson;

  /// 0..1
  final double mastery;
}
```

**ماذا يفعل؟** الحقول: المفهوم والدرس والإتقان.

**لماذا بهذه الطريقة؟** غير قابلة للتغيير.

الأسطر 61–67

```
/// «خطواتك الأولى»: ثلاث خطوات تُحسب من البيانات.
class FirstSteps {
  const FirstSteps({
    required this.downloaded,
    required this.practiced,
    required this.quizzed,
  });
```

**ماذا يفعل؟** صنف `FirstSteps` لـ«خطواتك الأولى».

**لماذا بهذه الطريقة؟** ثلاث علامات تُحسب من البيانات.

الأسطر 69–76

```
  /// نزّل (أو بدأ) أول درس.
  final bool downloaded;

  /// أكمل أول تدريب.
  final bool practiced;

  /// جرّب أول اختبار.
  final bool quizzed;
```

**ماذا يفعل؟** ثلاث علامات: نزّل/بدأ أول درس، أكمل أول تدريب، جرّب أول اختبار.

**لماذا بهذه الطريقة؟** كل خطوة تعني وجود بيانات تدل عليها.

الأسطر 78–79

```
  int get done => [downloaded, practiced, quizzed].where((d) => d).length;
}
```

**ماذا يفعل؟** `done`: عدد الخطوات المنجزة (0 إلى 3).

**لماذا بهذه الطريقة؟** «X من 3» في البطاقة.

الأسطر 81–94

```
/// كل ما تحتاجه الرئيسية، مشتقًا من قاعدة البيانات والكتالوج للملف النشط.
/// «أول يوم»: ملف جديد يعطي حالة ثابتة: لا يحتوي أي قيمة.
class HomeState {
  const HomeState({
    required this.profile,
    required this.hasActivity,
    required this.stars,
    required this.streak,
    required this.featured,
    required this.firstSteps,
    required this.subjects,
    required this.hasQuizAttempt,
    required this.review,
  });
```

**ماذا يفعل؟** تعليق وصنف `HomeState` وباني بكل الحقول.

**لماذا بهذه الطريقة؟** لا قيم ثابتة؛ ملف جديد يعطي حالة «أول يوم».

الأسطر 96–105

```
96    final Profile profile;
97
98    /// هل أنجز الطالب أي نشاط (محاولة أو بدء/تنزيل درس)؟
99    final bool hasActivity;
100
101   /// مجموع `starsEarned`.
102   final int stars;
103
104   /// أيام النشاط المتتالية حتى اليوم.
105   final int streak;
```

**ماذا يفعل؟** حقول: الملف وهل يوجد نشاط والنجوم والسلسلة.

**لماذا بهذه الطريقة؟** النجوم والسلسلة تُحسبان من البيانات لا تُخزَّنان.

الأسطر 107–113

```
107   /// الدرس المعروض في البطاقة الخضراء: أول درس (أول يوم) أو درس المتابعة.
108   final LessonRef? featured;
109
110   final FirstSteps firstSteps;
111   final List<SubjectProgress> subjects;
112   final bool hasQuizAttempt;
113   final List<ReviewItem> review;
```

**ماذا يفعل؟** حقول: الدرس المعروض والخطوات والمواد وهل وُجد اختبار وعناصر المراجعة.

**لماذا بهذه الطريقة؟** `featured` قد يكون `null` لو كان الكتالوج فارغًا.

السطر 115

```
115   bool get isFirstDay => !hasActivity;
```

**ماذا يفعل؟** `isFirstDay`: لا نشاط.

**لماذا بهذه الطريقة؟** يحدّد شكل الشاشة.

الأسطر 117–118

```
117   /// قسم «يحتاج إلى مراجعة» لا يظهر إلا بعد أول اختبار ووجود مفهوم ضعيف.
118   bool get showReviewSection => hasQuizAttempt && review.isNotEmpty;
```

**ماذا يفعل؟** `showReviewSection`: وجود اختبار و مفهوم ضعيف.

**لماذا بهذه الطريقة؟** القسم لا يظهر قبل أول اختبار حتى لو وُجد إتقان منخفض من مصدر آخر.

الأسطر 120–121

```
120   /// قبل أول نشاط يظهر صندوق منقّط يشرح متى ستظهر المراجعة.
121   bool get showReviewPlaceholder => isFirstDay;
```

**ماذا يفعل؟** `showReviewPlaceholder`: في أول يوم فقط.

**لماذا بهذه الطريقة؟** الصندوق المنقّط يشرح متى ستظهر المراجعة.

الأسطر 123–130

```
123   factory HomeState.derive({
124     required Profile profile,
125     required Catalog catalog,
126     required List<LessonProgressEntry> progress,
127     required List<Attempt> attempts,
128     required List<ConceptMasteryEntry> mastery,
129     required DateTime now,
130   }) {
```

**ماذا يفعل؟** المصنِّع `HomeState.derive` ومعاملاته: الملف والكتالوج والتقدّم والمحاولات والإتقان والوقت.

**لماذا بهذه الطريقة؟** دالة صرفة: لا قاعدة بيانات ولا ساعة داخلها (الوقت يُمرَّر) فتُختبر بدقة.

السطر 131

```
131     final progressByLesson = {for (final p in progress) p.lessonId: p};
```

**ماذا يفعل؟** خريطة التقدّم بمعرّف الدرس.

**لماذا بهذه الطريقة؟** بحث سريع عن حالة أي درس.

الأسطر 132–134

```
132     final started = progress
133         .where((p) => p.status != LessonStatus.notStarted)
134         .toList();
```

**ماذا يفعل؟** قائمة الدروس التي حالتها غير `notStarted`.

**لماذا بهذه الطريقة؟** هي «نشاط» الدروس.

السطر 136

```
136     final hasActivity = attempts.isNotEmpty || started.isNotEmpty;
```

**ماذا يفعل؟** `hasActivity`: محاولات موجودة أو درس بدأ.

**لماذا بهذه الطريقة؟** التعريف الوحيد للنشاط.

السطر 137

```
137     final stars = attempts.fold<int>(0, (sum, a) => sum + a.starsEarned);
```

**ماذا يفعل؟** النجوم: مجموع `starsEarned`.

**لماذا بهذه الطريقة؟** `fold` يجمع القيم.

الأسطر 138–141

```
    final streak = currentStreak([
      for (final a in attempts) a.createdAt,
      for (final p in started) p.updatedAt,
    ], now);
```

**ماذا يفعل؟** السلسلة من أوقات المحاولات وأوقات تحديث الدروس المبدوءة.

**لماذا بهذه الطريقة؟** أي نشاط في يوم يحسب لذلك اليوم.

الأسطر 143–151

```
    LessonRef refOf(CatalogLesson lesson) {
      final subject = catalog.subjectById(lesson.subjectId)!;
      final unit = catalog.unitOfLesson(lesson)!;
      final entry = progressByLesson[lesson.id];
      final fraction = switch (entry?.status) {
        LessonStatus.completed => 1.0,
        LessonStatus.inProgress => entry!.progress.clamp(0.0, 1.0),
        _ => 0.0,
      };
```

**ماذا يفعل؟** دالة محلية `refOf`: تجد مادة الدرس ووحدته وتحسب نسبته (مكتمل=1، قيد التقدّم=قيمته، غير ذلك 0).

**لماذا بهذه الطريقة؟** `clamp` يحمي من قيمة خارج 0..1.

الأسطر 152–158

```
      return LessonRef(
        lesson: lesson,
        subject: subject,
        unit: unit,
        progress: fraction,
      );
    }
```

**ماذا يفعل؟** نبني `LessonRef` ونغلق الدالة المحلية.

**لماذا بهذه الطريقة؟** تُستعمل لاحقًا للدرس المعروض.

السطر 160

```
    final featuredLesson = _pickFeatured(catalog, started, hasActivity);
```

**ماذا يفعل؟** نحدد الدرس المعروض بـ `_pickFeatured`.

**لماذا بهذه الطريقة؟** منطق مفصّل في الأسفل.

الأسطر 162–178

```
    final subjects = [
      for (final s in catalog.subjects)
        SubjectProgress(
          subject: s,
          total: s.lessons.length,
          completed: s.lessons
              .where(
                (l) => progressByLesson[l.id]?.status == LessonStatus.completed,
              )
              .length,
          started: s.lessons.any(
            (l) =>
                (progressByLesson[l.id]?.status ?? LessonStatus.notStarted) !=
                LessonStatus.notStarted,
          ),
        ),
    ];
```

**ماذا يفعل؟** لكل مادة نحسب: عدد دروسها والمكتمل منها وهل بدأ أي منها.

**لماذا بهذه الطريقة؟** تُعرض في «موادي».

الأسطر 180–183

```
    final review = <ReviewItem>[];
    final weak =
        mastery.where((m) => m.mastery < kReviewMasteryThreshold).toList()
          ..sort((a, b) => a.mastery.compareTo(b.mastery));
```

**ماذا يفعل؟** نجمع المفاهيم ذات الإتقان تحت العتبة ونرتّبها من الأضعف.

**لماذا بهذه الطريقة؟** `..sort` ترتّب القائمة في مكانها وتعيدها.

الأسطر 184–195

```
    for (final m in weak) {
      final hit = catalog.conceptById(m.conceptId);
      if (hit == null) continue;
      review.add(
        ReviewItem(
          concept: hit.concept,
          lesson: hit.lesson,
          mastery: m.mastery,
        ),
      );
      if (review.length == kMaxReviewItems) break;
    }
```

**ماذا يفعل؟** لكل مفهوم ضعيف نبحث عنه في الكتالوج؛ إن لم نجده نتجاهله، وإلا نضيف عنصر مراجعة ونتوقف عند الحد الأقصى.

**لماذا بهذه الطريقة؟** تجاهل المفاهيم المجهولة يحمي من بيانات قديمة.

الأسطر 197–202

```
    return HomeState(
      profile: profile,
      hasActivity: hasActivity,
      stars: stars,
      streak: streak,
      featured: featuredLesson == null ? null : refOf(featuredLesson),
```

**ماذا يفعل؟** نبني `HomeState`: الملف والنشاط والنجوم والسلسلة والدرس المعروض (إن وُجد).

**لماذا بهذه الطريقة؟** `featured` يبقى `null` إن لم يوجد درس.

الأسطر 203–207

```
      firstSteps: FirstSteps(
        downloaded: started.isNotEmpty,
        practiced: attempts.any((a) => a.kind == AttemptKind.practice),
        quizzed: attempts.any((a) => a.kind == AttemptKind.quiz),
      ),
```

**ماذا يفعل؟** خطوات البداية: بدأ درسًا، أجرى تدريبًا، أجرى اختبارًا.

**لماذا بهذه الطريقة؟** محسوبة بأسئلة بسيطة على المحاولات.

الأسطر 208–211

```
      subjects: subjects,
      hasQuizAttempt: attempts.any((a) => a.kind == AttemptKind.quiz),
      review: review,
    );
```

**ماذا يفعل؟** المواد وهل وُجد اختبار وعناصر المراجعة.

**لماذا بهذه الطريقة؟** ثم نغلق المصنّع.

السطر 212

```
  }
```

**ماذا يفعل؟** إغلاق `derive`.

**لماذا بهذه الطريقة؟** نهاية المصنّع.

الأسطر 214–217

```
  /// أي درس تعرضه البطاقة الخضراء؟
  /// - بلا نشاط: أول درس في الكتالوج.
  /// - وإلا: آخر درس `inProgress` (الأحدث تحديثًا)؛ وإن لم يوجد فآخر درس لُمس
  ///   إن لم يكتمل؛ وإن اكتمل فالدرس التالي في مادته؛ وإلا أول درس غير مكتمل.
```

**ماذا يفعل؟** تعليق قاعدة اختيار الدرس المعروض.

**لماذا بهذه الطريقة؟** بلا نشاط: أول درس. وإلا: أحدث `inProgress`؛ وإلا آخر درس لُمس إن لم يكتمل؛ وإن اكتمل فالتالي في مادته؛ وإلا أول غير مكتمل.

الأسطر 218–222

```
218   static CatalogLesson? _pickFeatured(
219     Catalog catalog,
220     List<LessonProgressEntry> started,
221     bool hasActivity,
222   ) {
```

**ماذا يفعل؟** دالة ساكنة `_pickFeatured(catalog, started, hasActivity)`.

**لماذا بهذه الطريقة؟** خاصة وصرفة.

السطر 223

```
223     if (!hasActivity || started.isEmpty) return catalog.firstLesson;
```

**ماذا يفعل؟** بلا نشاط ← أول درس.

**لماذا بهذه الطريقة؟** حالة «أول يوم».

الأسطر 225–226

```
225     int byRecent(LessonProgressEntry a, LessonProgressEntry b) =>
226         b.updatedAt.compareTo(a.updatedAt);
```

**ماذا يفعل؟** دالة مقارنة: الأحدث تحديثًا أولًا.

**لماذا بهذه الطريقة؟** تُستعمل في الترتيب.

الأسطر 228–230

```
228     final inProgress =
229         started.where((p) => p.status == LessonStatus.inProgress).toList()
230           ..sort(byRecent);
```

**ماذا يفعل؟** نجمع الدروس `inProgress` ونرتّبها بالأحدث.

**لماذا بهذه الطريقة؟** الدرس الذي كان الطالب يتعلّمه.

الأسطر 231–233

```
231     if (inProgress.isNotEmpty) {
232       return catalog.lessonById(inProgress.first.lessonId);
233     }
```

**ماذا يفعل؟** إن وُجد أي منها نعيد الأحدث.

**لماذا بهذه الطريقة؟** هذه بطاقة «متابعة التعلم».

الأسطر 235–237

```
    final recent = [...started]..sort(byRecent);
    final last = catalog.lessonById(recent.first.lessonId);
    if (last == null) return catalog.firstLesson;
```

**ماذا يفعل؟** نرتّب كل المبدوء بالأحدث ونأخذ آخر درس لُمس؛ إن لم يوجد في الكتالوج نعيد أول درس.

**لماذا بهذه الطريقة؟** حماية من درس حُذف من الكتالوج.

السطر 238

```
    if (recent.first.status != LessonStatus.completed) return last;
```

**ماذا يفعل؟** إن لم يكتمل آخر درس لُمس (مثل `downloaded`) نعيده.

**لماذا بهذه الطريقة؟** لا نقفز عن درس لم يبدأ فعليًا.

الأسطر 240–243

```
    final completedIds = {
      for (final p in started)
        if (p.status == LessonStatus.completed) p.lessonId,
    };
```

**ماذا يفعل؟** مجموعة معرّفات الدروس المكتملة.

**لماذا بهذه الطريقة؟** للبحث السريع.

الأسطر 244–245

```
    final next = catalog.nextLessonInSubject(last);
    if (next != null && !completedIds.contains(next.id)) return next;
```

**ماذا يفعل؟** إن اكتمل آخر درس: التالي في المادة إن لم يكتمل.

**لماذا بهذه الطريقة؟** استمرار طبيعي.

الأسطر 247–249

```
    for (final lesson in catalog.allLessons) {
      if (!completedIds.contains(lesson.id)) return lesson;
    }
```

**ماذا يفعل؟** وإلا أول درس غير مكتمل في الكتالوج.

**لماذا بهذه الطريقة؟** نبحث بترتيب الكتالوج.

الأسطر 250–252

```
    return last;
  }
}
```

**ماذا يفعل؟** وإن اكتمل كل شيء نعيد آخر درس ثم إغلاق الصنف.

**لماذا بهذه الطريقة؟** حتى لا نعيد `null` لطالب أنهى كل شيء.

# lib/features/home/home_providers.dart — مزوّدات الرئيسية

lib/features/home/home_providers.dart

الأسطر 1–7

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/app_database.dart';
import '../../core/db/database_provider.dart';
import '../catalog/catalog_providers.dart';
import '../profiles/active_profile_provider.dart';
import 'home_state.dart';
```

**ماذا يفعل؟** استيراد Riverpod والقاعدة ومزوّدها والكتالوج والملف النشط وحالة الرئيسية.

**لماذا بهذه الطريقة؟** يجمع المزوّد مصادر متعددة.

الأسطر 9–15

```dart
final lessonProgressProvider = StreamProvider.autoDispose
    .family<List<LessonProgressEntry>, String>(
      (ref, profileId) => ref
          .watch(appDatabaseProvider)
          .activityDao
          .watchLessonProgress(profileId),
    );
```

**ماذا يفعل؟** `lessonProgressProvider`: بث بتقدّم دروس ملف (`family` بمعرّف الملف).

**لماذا بهذه الطريقة؟** `autoDispose` يتخلّص منه عند إغلاق الشاشة. `family` لأنه يعتمد على معرّف الملف.

الأسطر 17–21

```dart
final attemptsProvider = StreamProvider.autoDispose
    .family<List<Attempt>, String>(
      (ref, profileId) =>
          ref.watch(appDatabaseProvider).activityDao.watchAttempts(profileId),
    );
```

**ماذا يفعل؟** `attemptsProvider`: بث بمحاولات الملف.

**لماذا بهذه الطريقة؟** مصدر النجوم والسلسلة والخطوات.

الأسطر 23–27

```dart
final masteryProvider = StreamProvider.autoDispose
    .family<List<ConceptMasteryEntry>, String>(
      (ref, profileId) =>
          ref.watch(appDatabaseProvider).activityDao.watchMastery(profileId),
    );
```

**ماذا يفعل؟** `masteryProvider`: بث بإتقان مفاهيم الملف.

**لماذا بهذه الطريقة؟** مصدر قسم المراجعة.

**الأسطر 29–31**

```
/// حالة الرئيسية للملف النشط، تُعاد تلقائيًا عند تغيّر أي مصدر من مصادرها.
/// `AsyncData(null)` = لا يوجد ملف نشط.
final homeStateProvider = Provider.autoDispose<AsyncValue<HomeState?>>((ref) {
```

**ماذا يفعل؟** تعليق و`homeStateProvider`: مزوّد يجمع كل المصادر في `AsyncValue<HomeState?>`.

**لماذا بهذه الطريقة؟** `AsyncData(null)` تعني لا يوجد ملف نشط.

**الأسطر 32–33**

```
  final profileAsync = ref.watch(activeProfileProvider);
  final catalogAsync = ref.watch(catalogProvider);
```

**ماذا يفعل؟** نراقب الملف النشط والكتالوج.

**لماذا بهذه الطريقة؟** `watch` فيُعاد الحساب عند تغيّر أي منهما.

**الأسطر 35–40**

```
  if (profileAsync.hasError) {
    return AsyncError(profileAsync.error!, profileAsync.stackTrace!);
  }
  if (catalogAsync.hasError) {
    return AsyncError(catalogAsync.error!, catalogAsync.stackTrace!);
  }
```

**ماذا يفعل؟** إن كان أحدهما بخطأ نعيد الخطأ.

**لماذا بهذه الطريقة؟** الشاشة تعرض رسالة بدل الانهيار.

**الأسطر 42–47**

```
  final profile = profileAsync.value;
  if (profile == null) {
    return profileAsync.isLoading
        ? const AsyncLoading()
        : const AsyncData(null);
  }
```

**ماذا يفعل؟** إن لم يوجد ملف: `Loading` إن كنا ما زلنا نحمّل وإلا `Data(null)`.

**لماذا بهذه الطريقة؟** نفرّق بين «لم يصل بعد» و«لا يوجد».

**الأسطر 49–52**

```
  final catalog = catalogAsync.value;
  final progress = ref.watch(lessonProgressProvider(profile.id));
  final attempts = ref.watch(attemptsProvider(profile.id));
  final mastery = ref.watch(masteryProvider(profile.id));
```

**ماذا يفعل؟** نراقب تقدّم الدروس والمحاولات والإتقان لهذا الملف.

**لماذا بهذه الطريقة؟** ثلاثة بثّات.

الأسطر 54–58

```
    for (final source in [progress, attempts, mastery]) {
      if (source.hasError) {
        return AsyncError(source.error!, source.stackTrace!);
      }
    }
```

**ماذا يفعل؟** إن كان أيّ مصدر بخطأ نعيده.

**لماذا بهذه الطريقة؟** مرور سريع على الثلاثة.

الأسطر 59–64

```
    if (catalog == null ||
        progress.value == null ||
        attempts.value == null ||
        mastery.value == null) {
      return const AsyncLoading();
    }
```

**ماذا يفعل؟** إن كان أيّ مصدر لم يصل بعد نعيد `Loading`.

**لماذا بهذه الطريقة؟** لا نبني حالة ناقصة.

الأسطر 66–76

```
    return AsyncData(
      HomeState.derive(
        profile: profile,
        catalog: catalog,
        progress: progress.value!,
        attempts: attempts.value!,
        mastery: mastery.value!,
        now: ref.watch(clockProvider)(),
      ),
    );
});
```

**ماذا يفعل؟** نبني `HomeState` بكل المصادر والساعة الحالية ونعيدها `Data`، ثم إغلاق المزوّد.

**لماذا بهذه الطريقة؟** الساعة من `clockProvider` فيمكن ضبطها في الاختبارات.